

Dev Legacy — Compte-rendu de mission

Maintenance & modernisation — Unify

Projet	Unify — plateforme sociale de running (React Native / Expo + microservice NestJS)
Équipe	Tim Hrdy — Chantiers 1 & 2, rapport · <i>[coéquipier — rôle à compléter]</i> — Chantier 3
Dépôt	https://github.com/guiguil03/Unify-App
Branche	features/post-profil
Pull Request	 <i>[lien PR à insérer — accès lecture @celianlb]</i>
Dates d'intervention	5-6 juin et 12 juin 2026

1. Reprise en main

État du projet au retour. Monorepo à deux têtes : l'app mobile Expo SDK 55 / React Native 0.83 à la racine, et un microservice NestJS 10 (`micro-affil/`) en Clean Architecture. Le backend de données est Supabase.

Temps pour le faire tourner : ~15 min pour l'app (`npm install + expo start` , rien de cassé côté lancement) ; le microservice **ne compilait pas** (voir Chantier 2).

- Frictions rencontrées à la reprise :**
- **Build du microservice cassé :** `npm run build` échoue (`error TS5095`) — c'est devenu notre Chantier 2.
 - **Lint du microservice cassé :** 21 erreurs de parsing (ESLint tentait d'analyser les `.d.ts` générés dans `dist/`).
 - **Dépendance morte :** `firebase` déclaré dans `package.json` mais **importé nulle part** (l'app utilise exclusivement Supabase) — et c'est lui qui portait la faille **critique** `protobufjs` .
 - **Dépendances parasites :** deux librairies React Native (`@react-navigation/bottom-tabs` , `@react-native-community/datetimepicker`) déclarées... dans le backend NestJS, sans aucune référence dans `src/` .
 - **Artefact de build versionné :** un dossier `Unify.app/` (binaire iOS compilé, ~70 fichiers) committé dans le dépôt par erreur.
 - `.env` requis pour lancer le microservice (variables Supabase) — non versionné, à recréer.

 Capture à insérer : le projet qui démarre (terminal `expo start` + simulateur affichant l'écran d'accueil)

2. Tableau de bord « avant »

Mesures prises **avant toute intervention** (5 juin 2026) :

Indicateur	App mobile (racine)	Microservice <code>micro-affil</code>
Vulnérabilités (<code>npm audit</code>)	24 (1 critique, 6 élevées, 17 modérées)	36 (1 critique, 17 élevées, 14 modérées, 4 faibles)
Dépendances obsolètes (<code>npm outdated</code>)	40+ paquets en retard (dont React Navigation v6 → v7, ESLint 8 → 10)	NestJS 10 (latest : 11), <code>@typescript-eslint</code> 6 (latest : 8)
Build / lint OK ?	Tests OK · lint non bloquant	Build KO (TS5095) · Lint KO (21 erreurs)
Nb de tests / couverture	14 tests, 4 suites (filet existant)	7 tests, 2 suites · couverture 100 % des unités testées
Temps de réponse clé (back)	—	<i>non mesuré (env Supabase requis)</i>

Sortie `npm audit` initiale (extraits réels) :

```
# App mobile (racine)
24 vulnerabilities (17 moderate, 6 high, 1 critical)

protobufjs <=7.5.7 [CRITIQUE]
  * Arbitrary code execution in protobufjs
  * Prototype injection in generated message constructors
  via: firebase → @firebase/firestore → @grpc/proto-loader → protobufjs

lodash <=4.17.23 [ÉLEVÉE]
  * Code Injection via `_.template`
  * Prototype Pollution via `_.unset` / `_.omit`

# Microservice micro-affil
36 vulnerabilities (4 low, 14 moderate, 17 high, 1 critical)
  multer / express / body-parser / qs [ÉLEVÉES] — pile HTTP NestJS 10
```

 Capture à insérer : sortie complète de `npm audit` et `npm outdated` avant intervention (terminal)

3. Fiche de cadrage

Chantier	Détail	« Fait » = quand...
C1 — Mise à jour	NestJS 10 → 11 (majeure, breaking changes Express 5); suppression <code>firebase</code> (dépendance morte portant la faille critique); <code>lodash</code> 4.17.23 → 4.18.1; <code>@typescript-eslint</code> 6 → 8; correctifs lockfile (<code>node-forge</code> , <code>xmldom</code> , <code>shell-quote</code> ...). Plan de rollback : §C1.4	<code>npm audit</code> = 0 critique / 0 élevée sur les deux projets ; build + 14 tests verts
C2 — Correctif	Build du microservice cassé : <code>npm run build</code> → <code>error TS5095</code> . Fichiers : <code>micro-affil/tsconfig.json</code> , <code>micro-affil/.eslintrc.js</code>	<code>tsc</code> exit 0 ; lint 0 erreur ; tests verts
C3 — Évolutif	<i>[coéquipier]</i> — deep linking d'authentification + inscription via Edge Function Supabase. Fichiers : <code>App.tsx</code> , <code>AuthService</code> , <code>AuthContext</code>	Flux d'auth Google fonctionnel via deep link, testé sur device

Répartition : Tim Hrdy → C1 + C2 + rapport · *[coéquipier]* → C3.

4.5 Preuve de non-régression

```
$ npm run build      (micro-affil)
> tsc
BUILD_EXIT=0 - dist/src/main.js généré

$ npm test           (racine, après toutes les montées)
Test Suites: 4 passed, 4 total
Tests:       14 passed, 14 total

$ npm run test:cov    (micro-affil)
All files       | 100 % stmts | 100 % branch | 100 % funcs | 100 % lines
Tests:         7 passed, 7 total
```

Gain quantifié : 60 → 9 vulnérabilités (-85 %), **0 critique / 0 élevée** ; 2 failles critiques d'exécution de code éliminées ; bundle mobile allégé du SDK Firebase complet (inutilisé).

 Capture à insérer : tests au vert + `npm audit` après, dans le terminal

5. C2 — Correctif : build du microservice cassé

Symptôme. Le microservice ne compile pas :

```
$ cd micro-affil && npm run build
> tsc
tsconfig.json(2,3): error TS5095: Option 'bundler' can only be used when
'module' is set to 'preserve' or to 'es2015' or later.
```

Reproduction. Systématique : `npm run build` échoue à chaque exécution, avant toute modification de code. (Le service tournait jusqu'ici en dev via `ts-node-dev --transpile-only`, qui ignore les erreurs de type — c'est pour ça que personne ne l'avait vu.)

Diagnostic — cause racine. Le `tsconfig.json` du backend se terminait par :

```
"extends": "expo/tsconfig.base"
```

Un backend NestJS (CommonJS) qui hérite de la config TypeScript d'Expo (front React Native). Cette base impose `moduleResolution: "bundler"` et `customConditions` — incompatibles avec `module: "commonjs"`. Probablement un copier-coller à la création du dossier dans le monorepo. Première correction tentée (`moduleResolution: "node"` seul) → nouvelle erreur TS5098 (`customConditions` hérité lui aussi) : traiter le symptôme ne suffisait pas, il fallait **supprimer l'héritage**, pas le rustiner.

Correction (`micro-affil/tsconfig.json`) :

```
"compilerOptions": {
  "module": "commonjs",
+  "moduleResolution": "node",
  ...
},
- "extends": "expo/tsconfig.base"
}
```

Dans la foulée, même famille de problème sur le lint : ESLint analysait les `.d.ts` générés dans `dist/` (21 erreurs de parsing) → exclusion de `dist/` et `coverage/` dans `.eslintrc.js`.

Test de non-régression.

```
$ npm run build → exit 0, dist/src/main.js généré
$ npm run lint → 0 erreur (21 → 0)
$ npm test → 14/14 verts
```

La compilation `tsc` (vérification de types complète, vs `--transpile-only` en dev) et le lint font désormais office de **garde-fou permanent** : toute réintroduction du problème casse le build immédiatement.

 Capture à insérer : avant/après dans le terminal (erreur TS5095 → build vert)

6. C3 — Évolutif : deep linking d'authentification

 **Section à compléter par son auteur** (répartition d'équipe — voir fiche de cadrage).

Besoin. Finaliser le flux d'authentification Google : au retour du navigateur OAuth, l'app doit récupérer la session via **deep link** au lieu de laisser l'utilisateur sur un écran mort ; l'inscription doit passer par une **Edge Function** Supabase pour gérer les limites de débit côté serveur.

Implémentation. Gestion des deep links dans `App.tsx` ; refactor de `AuthService.register()` vers l'Edge Function ; gestion d'erreurs renforcée dans `AuthContext` (commits `efaad33` et PR #9 `fix-google-auth`).

Test. [à compléter : scénario de test du flux complet — login Google → retour app → session active]

 Captures à insérer : avant (retour OAuth sans session) / après (deep link → utilisateur connecté)

7. Tableau de bord « après » + hygiène Git

Indicateur	App mobile (racine)	Microservice <code>micro-affil</code>
Vulnérabilités (<code>npm audit</code>)	9 modérées (0 critique, 0 élevée) — 24 → 9	0 — 36 → 0
Dépendances obsolètes	41 paquets* (écosystème Expo 55, voir §8)	18 (transitifs mineurs)
Build / lint OK ?	Tests OK	Build OK · Lint OK (réparés)
Nb de tests / couverture	14/14 verts (inchangé = non-régression)	7/7 verts · couverture 100 % des unités testées
Temps de réponse clé (back)	—	<i>non mesuré (env Supabase requis)</i>

* Les 9 vulnérabilités modérées restantes sont **toutes** dans l'outillage de build Expo (`@expo/cli` , `xcode` , `metro-config` ...), non embarqué dans l'app livrée. Le seul « fix » proposé par npm est un **downgrade Expo 55** → **46** (`isSemVerMajor`), refusé car destructeur. Vraie remédiation : **migration Expo SDK 56**, planifiée comme chantier dédié.

```
$ npm audit # racine, après
9 moderate severity vulnerabilities

$ cd micro-affil && npm audit # microservice, après
found 0 vulnerabilities
```

Hygiène Git — branche dédiée `features/post-profil` , commits atomiques (1 commit = 1 intention) :

```
* b74f6f7 chore(micro-affil): exclude dist/ et coverage/ du lint
* f96ce87 security: fix critique shell-quote (GHSA-w7jw-789q-3m8p) via npm audit fix
* 430203b security: audit et correction des vulnérabilités des dépendances
* d9944a8 Merge pull request #9 from guiguil03/fix-google-auth
| * fc8ec92 fix auth
* | efaad33 Add deep link handling for authentication in App component; ...
```

Captures à insérer : `npm audit` après (les deux projets) + page de la PR GitHub avec l'historique des commits

8. Bilan & rétro legacy

Ce que les vieilles deps / le vieux code ont coûté :

- La faille **critique** (exécution de code arbitraire via `protobufjs`) venait d'un SDK **que personne n'utilisait**. Firebase a été ajouté un jour, jamais branché, jamais retiré : chaque dépendance « au cas où » est une dette de sécurité qui court toute seule.
- Le build du microservice était cassé **silencieusement** depuis sa création : le mode dev (`--transpile-only`) masquait l'erreur de config. Sans CI qui compile, un projet peut être indéployable sans que personne ne le sache.
- Une **nouvelle faille critique est apparue entre nos deux sessions** (`shell-quote` , advisory du 12/06) : la sécurité des dépendances n'est pas un état, c'est un flux. Un audit ponctuel périmé en jours.

Ce qu'on change dans nos habitudes :

- `npm audit --audit-level=high` **bloquant en CI** + compilation `tsc` du microservice à chaque PR (plus jamais de build cassé silencieux).
- Dependabot/Renovate** sur le dépôt : les correctifs de sécurité arrivent en PR automatiques, petits et fréquents, au lieu d'un rattrapage massif et risqué.
- Une **dépendance ajoutée = une dépendance justifiée** ; revue trimestrielle de `npm outdated` pour ne plus accumuler 40 paquets de retard.
- Ne pas mélanger les intentions dans un commit** : notre premier commit (430203b) regroupait l'upgrade C1 et le correctif C2 — les deux suivants sont redevenus atomiques. Leçon intégrée en cours de mission.
- Sortir les artefacts de build (`Unify.app/` , `dist/`) du dépôt et du lint : `.gitignore` à compléter.

Prochaine échéance identifiée : migration **Expo SDK 56** (efface les 9 vulnérabilités résiduelles) — à mener avec la même méthode : filet, changelog, branche dédiée, rollback.

Annexe A — Journal de bord

Horodatage	Événement
05/06 ~23h15	Reprise du projet. <code>npm audit</code> : 24 vulnérabilités app (1 critique) / 36 micro (1 critique). <code>npm outdated</code> : 40+ paquets en retard.
05/06 ~23h30	Filet posé : <code>npm test</code> → 14/14 verts (comportement de référence figé).
05/06 ~23h40	<code>npm audit fix --dry-run</code> sur les deux projets pour trier correctifs sûrs / cassants. Décision : refuser <code>--force</code> (proposait un downgrade Expo 55 → 46).
05/06 ~23h50	<code>npm audit fix</code> racine : 24 → 10. Critique <code>protobufjs</code> et toutes les élevées corrigées via lockfile.
06/06 ~00h05	micro-affil : lecture du guide de migration NestJS 11 (Node ≥ 20 ✓, Express 5, routes wildcard nommées — non concernés). Suppression des 2 deps RN parasites. Montée NestJS 10 → 11 + <code>@typescript-eslint</code> 8.
06/06 ~00h15	Impasse n°1 : <code>npm run build</code> → <code>error TS5095</code> . Première piste (<code>moduleResolution: "node"</code>) → nouvelle erreur <code>TS5098</code> .
06/06 ~00h25	Cause racine trouvée : <code>extends: "expo/tsconfig.base"</code> dans le <code>tsconfig</code> du backend. Suppression de l'héritage → build vert. Le service ne compilait en fait jamais (<code>--transpile-only</code> masquait tout).
06/06 ~00h35	Dernier <code>npm audit fix</code> micro (<code>glob</code>) → 0 vulnérabilité . Re-build + re-test : verts.
06/06 ~00h45	Question d'équipe : « on n'utilise pas Firebase, si ? » → vérif par grep : aucun import . Suppression de <code>firebase</code> → sous-arbre <code>protobufjs</code> / <code>grpc</code> éliminé à la racine. Tests verts.
06/06 00h59	Commit <code>430203b</code> (audit + correctifs) — <i>rétrospectivement trop gros : mélange C1/C2, voir rétro</i> . Push.
12/06 ~02h00	Relecture de l'énoncé exact. Re-audit de contrôle : nouvelle faille critique apparue entre-temps (<code>shell-quote</code> , GHSA-w7jw-789q-3m8p, advisory publié après notre session).
12/06 02h17	Fix <code>shell-quote</code> 1.8.3 → 1.8.4 (<code>npm audit fix</code> , non-cassant), tests verts. Commit atomique <code>f96ce87</code> .
12/06 ~02h25	Lint micro-affil : 21 erreurs de parsing (<code>dist/</code> analysé par ESLint). Exclusion <code>dist/</code> + <code>coverage/</code> → 0 erreur. Commit atomique <code>b74f6f7</code> . <code>npm run test:cov</code> : 100 % sur les unités testées.
12/06 ~02h40	Refonte du compte-rendu selon le canevas du sujet.

Annexe B — Référence

Rapport d'audit détaillé (CVE par CVE, CVSS, CWE) : `docs/AUDIT-SECURITE.md` dans le dépôt.